PAL: Program-aided Language Models

How to connect the weaknesses of LLMs in performing algorithmic executions

Eric Peter

Heidelberg University

21. May 2026

Contents

Part	Goal
1. Motivation and background	LLMs cannot calculate consistently; few-shot and chain-of-thought prompting
2. Core idea of PAL	Explain how programs become intermediate reasoning steps and why the interpreter matters
3. Experimental setup	Tasks, baselines, models, and evaluation protocol
4. Results and analysis	Main quantitative results, robustness, ablations, and failure modes
5. Discussion and conclusion	Strengths and limitations

Motivation: LLMs are bad calculators

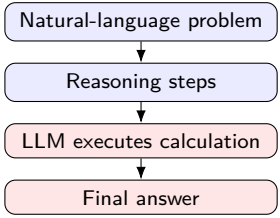
Observation behind PAL

Large language models can often translate a natural-language problem into useful intermediate steps, but may still make arithmetic or logical execution mistakes.

- Few-shot prompting shows examples at test time.
- Chain-of-thought prompting asks the model to write intermediate reasoning steps.
- But in CoT, the same model both *plans* and *executes* the solution.

Problem

Even if the decomposition is plausible, the final answer can be wrong because the LLM performs calculations as text generation rather than exact computation.



Background 1: few-shot and chain-of-thought prompting

Definition

Both techniques provide the LLM a pre-prompt $\mathbf{p}$ which appended to the question $\mathbf{x}$ is supposed to achieve a more precise output $\mathbf{y}$.
In particular $\mathbf{p}$ provides a representation of $\mathbf{k}$ question-answer-tuples $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{k \in \mathbb{N}}$ which are supposed to tell the LLM how to derive his solutions.

few-shot	**CoT**
$\mathbf{p} \equiv \langle \mathbf{x}_1 \cdot \mathbf{y}_1 \rangle \parallel \langle \mathbf{x}_2 \cdot \mathbf{y}_2 \rangle \parallel \cdots \parallel \langle \mathbf{x}_k \cdot \mathbf{y}_k \rangle$ Then $\mathbf{p} \parallel \mathbf{x} \underset{\text{LLM}}{\rightarrow} \mathbf{y}$	$\mathbf{p} \equiv \langle \mathbf{x}_1 \cdot \mathbf{t}_1 \cdot \mathbf{y}_1 \rangle \parallel \langle \mathbf{x}_2 \cdot \mathbf{t}_2 \cdot \mathbf{y}_2 \rangle \parallel \cdots \parallel \langle \mathbf{x}_k \cdot \mathbf{t}_k \cdot \mathbf{y}_k \rangle$ Then $\mathbf{p} \parallel \mathbf{x} \underset{\text{LLM}}{\rightarrow} \mathbf{t} \cdot \mathbf{y}$

Where $\cdot$ and $\parallel$ are appropriate separators in order to represent $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{k \in \mathbb{N}}$.
$\rightarrow$ CoT extends *few-shot* by providing an **additional derivation** $\mathbf{t}_i$.
Then $\mathbf{p}$ is a representation of $\{(\mathbf{x}_i, \mathbf{t}_i, \mathbf{y}_i)\}_{k \in \mathbb{N}}$.

Background 2: few-shot and chain-of-thought prompting

few-shot	**CoT**
$\mathbf{p} \equiv \langle \mathbf{x}_1 \cdot \mathbf{y}_1 \rangle \parallel \langle \mathbf{x}_2 \cdot \mathbf{y}_2 \rangle \parallel \cdots \parallel \langle \mathbf{x}_k \cdot \mathbf{y}_k \rangle$ Then $\mathbf{p} \parallel \mathbf{x} \underset{\text{LLM}}{\rightarrow} \mathbf{y}$	$\mathbf{p} \equiv \langle \mathbf{x}_1 \cdot \mathbf{t}_1 \cdot \mathbf{y}_1 \rangle \parallel \langle \mathbf{x}_2 \cdot \mathbf{t}_2 \cdot \mathbf{y}_2 \rangle \parallel \cdots \parallel \langle \mathbf{x}_k \cdot \mathbf{t}_k \cdot \mathbf{y}_k \rangle$ Then $\mathbf{p} \parallel \mathbf{x} \underset{\text{LLM}}{\rightarrow} \mathbf{t} \cdot \mathbf{y}$
Example question $\mathbf{x}_i$: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?	
Answer $\mathbf{y}_i$: 11	Answer $\mathbf{t}_i \cdot \mathbf{y}_i$: Roger started with 5 tennis balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Where $\cdot$ and $\parallel$ are appropriate separators in order to represent $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{k \in \mathbb{N}}$.
$\rightarrow$ CoT extends *few-shot* by providing an **additional derivation** $\mathbf{t}_i$.
Then $\mathbf{p}$ is a representation of $\{(\mathbf{x}_i, \mathbf{t}_i, \mathbf{y}_i)\}_{k \in \mathbb{N}}$.

Why Chain-of-Thought is not enough?

CoT improves performance over direct answering. Especially in BBH-benchmarks but... ...

- biased prompts still influence the LLMs output.
- even with unbiased prompts, the model reaches it's limitations
- problem: the model still has to do two different things at once:
  1. decompose the problem into useful steps,
  2. execute these steps correctly.

Where the limitation appears

- At a certain amount of complexity, e.g. number of objects involved into the problem, the precision drops sharply
- When calculating with large numbers the LLM often gets wrong results
- The reasoning trace may look plausible, but the final answer can still be wrong because the model makes arithmetic, logical, or bookkeeping errors.

Core idea of PAL

The key problem

With CoT we can split problems into smaller subproblems correctly.
But when "thinking" about too many objects or dealing with arithmetically large numbers, the LLM becomes inconsistent:

- LLM trades off higher complexity with consistent algorithmic execution
- LLM performs rather textbased than algorithmic calculations
- good algorithms have never dealt with such kind of problems

Solution

Don't let the model perform calculations or algorithmic steps.
Let **only generate the code** which outputs the solution an run with **Python**.

→ relieve the model of the expensive work on execution steps

→ let this work be done by the interpreter

Why **Python**? It's simple. No need to expensively deal with pointer, references etc. which are not relevant for the problem we want to solve.

What changes?

CoT	PAL
$\mathbf{p} = \langle \mathbf{x}_1 \cdot \mathbf{t}_1 \cdot \mathbf{y}_1 \rangle \,\|\, \langle \mathbf{x}_2 \cdot \mathbf{t}_2 \cdot \mathbf{y}_2 \rangle \,\|\, \cdots \,\|\, \langle \mathbf{x}_k \cdot \mathbf{t}_k \cdot \mathbf{y}_k \rangle$ Then $\mathbf{p} \,\|\, \mathbf{x} \underset{\text{LLM}}{\rightarrow} \mathbf{t} \cdot \mathbf{y}$	$\mathbf{p} \equiv \langle \mathbf{x}_1 \cdot \mathbf{t}_1 \rangle \,\|\, \langle \mathbf{x}_2 \cdot \mathbf{t}_2 \rangle \,\|\, \cdots \,\|\, \langle \mathbf{x}_k \cdot \mathbf{t}_k \rangle$ Then $\mathbf{p} \,\|\, \mathbf{x} \underset{\text{LLM}}{\rightarrow} \mathbf{t} \underset{\text{Python}}{\rightarrow} \mathbf{y}$

Example question $\mathbf{x}_i$:
Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

Answer $\mathbf{t}_i \cdot \mathbf{y}_i$:	Answer $\mathbf{t}_i$: (no $\mathbf{y}_i$!)

Roger started with 5 tennis balls.
2 cans of 3 tennis balls each is 6 tennis balls.
$5 + 6 = 11$. The answer is 11.
	`tennis_balls = 5`
	`bought_balls = 2 * 3`
	`answer = tennis_balls + bought_balls`

Where $\mathbf{t}_i = [\mathbf{s}_1, \mathbf{s}_2, ..., \mathbf{s}_N]$ with $\mathbf{s}_j \in \text{NL} \cup \text{PL}$

$\mathbf{s}_j$ is either a comment in natural language (NL) or programming language (PL).

Example of $\langle \mathbf{x}_i \cdot \mathbf{t}_i \rangle$: symbolic reasoning with PAL

If $\mathbf{x}_i =$

On the table, you see a bunch of objects arranged in a row:

a purple paperclip →a pink stress ball →a brown keychain
→a green phone charger →a mauve fidget spinner →a burgundy pen

What is the color of the object directly to the right of the stress ball?

Then $\mathbf{t}_i =$

```
# Put objects into a list to record ordering -> NL
objects = [('paperclip', 'purple'), ('stress ball', 'pink'), ('keychain', 'brown'),
      ('scrunchiephone charger', 'green'), ('fidget spinner', 'mauve'), ('pen', 'burgundy')]
# Find the index of the stress ball -> NL
stress_ball_idx = None
for i, obj in enumerate(objects):
    if obj[0] == 'stress ball':
        stress_ball_idx = i
        break
# Find the directly right object -> NL (omit index check here)
direct_right = objects[stress_ball_idx + 1]
# Get the directly right object's color -> NL
answer = direct_right[1]
```

Example of $\langle \mathbf{x}_i \cdot \mathbf{t}_i \rangle$: mathematical reasoning with PAL

Question $\mathbf{x}_i \downarrow$ **Then $\mathbf{t}_i \rightarrow$**

Olivia has $23. She bought five bagels for $3 each. How much money does she have left?

```
money_initial = 23
bagels = 5
bagel_cost = 3
money_spent = bagels * bagel_cost
money_left = money_initial - money_spent
answer = money_left
```

Note

1. Direct example results $\mathbf{y}_i$ are not included in $\mathbf{p}$ (*see Code above*) and
2. the output $\mathbf{y}$ should not be generated by the LLM eitherway.

For the final output we have to perform this transition

$$\mathbf{p} \parallel \mathbf{x} \underset{\text{LLM}}{\rightarrow} \mathbf{t} \underset{\text{Python}}{\rightarrow} \mathbf{y}.$$

Let's generate an output!

Experimental setup

Task families

- Mathematical word problems
- Symbolic reasoning
- Algorithmic tasks

Baselines

- DIRECT prompting
- COT prompting
- PAL prompting

Main backend

- Codex / code-davinci-002
- Greedy decoding, temperature 0
- Accuracy / solve rate

Why this setup matters

The experiments test whether the gain comes from program-aided execution rather than simply from using a larger language model.

Benchmarks used in the paper

Category	Benchmark	Measures
Mathematical	GSM8K	Multi-step grade-school math word problems.
	GSM-HARD	Harder GSM8K-style arithmetic problems.
	SVAMP	Robustness on varied arithmetic word problems.
	ASDIV	Diverse arithmetic word-problem solving.
	SINGLEEQ	Single-equation math word problems.
	SINGLEOP	Single-operation arithmetic problems.
	ADDSUB	Addition and subtraction word problems.
	MULTIARITH	Multi-step arithmetic word problems.
Symbolic	COLORED OBJECTS	Reasoning over object-color relations.
	PENGUINS	Reasoning over table-based attributes.
	DATE	Reasoning over dates and calendars.
Algorithmic	OBJECT COUNTING	Counting specified objects.
	REPEAT COPY	Following copy/repetition instructions.

Table: Benchmarks used in PAL.

Category	Benchmarks / examples

GSM8K/GSM-HARD and COLORED OBJECTS/OBJECT COUNTING

Presentation focus

I will focus on **GSM8K/GSM-HARD** for arithmetic robustness, and on **COLORED OBJECTS/OBJECT COUNTING** for symbolic and algorithmic reasoning.

Benchmark	Example question
GSM8K/GSM-HARD	Olivia has **$23**. She bought **five** bagels for **$3** each. How much money does she have left?
COLORED OBJECTS	On the nightstand, there is a red red pencil, a purple mug, a burgundy keychain, a fuchsia teddy bear, a black plate, and a blue stress ball. What color is the stress ball?
OBJECT COUNTING	I have **one** chair, **two** potatoes, **one** cauliflower, **a** lettuce head, **two** tables, **one** cabbage, **two** onions, and **three** fridges. How many vegetables do I have?

Main math results: PAL improves across datasets

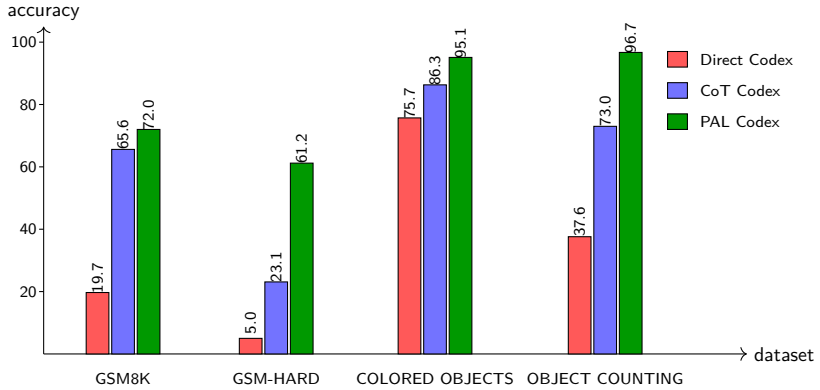
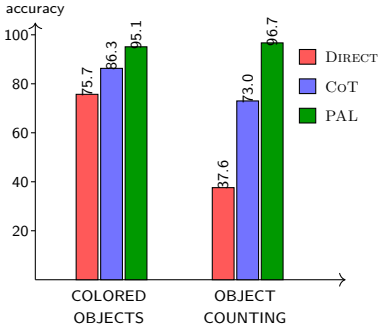


Figure: PAL paper results on arithmetic, symbolic, and algorithmic reasoning benchmarks.

The largest difference appears on **GSM-HARD**: when numbers become large, **CoT** accuracy drops sharply, while **PAL** remains more stable because **Python performs the arithmetic**.

Robustness Result: COLORED OBJECTS/OBJECT COUNTING



Source: PAL paper, Table 2 and Section 5.2.

Observation

On both selected symbolic/algorithmic tasks, PAL performs best.

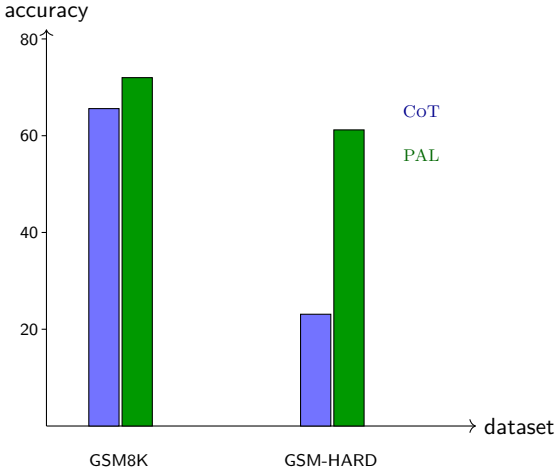
Why PAL helps

These tasks can be translated into explicit program operations: dictionaries for object–color lookup, lists for ordering, and counting/filtering loops for object categories.

Why the gain is smaller

The tasks are less arithmetically fragile than GSM-HARD: CoT can already solve many cases by tracking the objects in text, so PAL mainly reduces bookkeeping errors rather than fixing difficult calculations.

Robustness result: GSM8K vs. GSM-HARD



Source: PAL paper, Table 1 and Section 5.1.

GSM-HARD construction

The authors create a harder variant of GSM8K by replacing numbers in the questions with larger numbers, including values up to seven digits.

Observed failure mode

For many examples, CoT produces nearly the same reasoning text for the original and hard versions, suggesting that the main problem is arithmetic execution rather than decomposition.

Analysis: what actually makes PAL work?

Interpreter matters

When the model generated Python-like code but had to "execute" it itself, performance on GSM8K was far below full PAL.

Variable names matter

Replacing meaningful variable names with random names reduced accuracy; names help ground program variables to problem entities.

Not restricted to code-only LMs

The paper reports that sufficiently capable natural-language LMs can also benefit from PAL, although weaker models may struggle with code generation.

Runtime errors are rare

The generated Python code was syntactically correct, and fewer than 1% of examples raised runtime exceptions.

Strengths and limitations

Strengths

- Clear neuro-symbolic division of labor.
- Strong gains on math, symbolic, and algorithmic tasks.
- Robustness to larger numbers.
- Simple implementation: prompt + interpreter.

Limitations / questions

- Correctness still depends on generating the right program.
- Execution safety and sandboxing are not the central focus.
- Some tasks may not map cleanly to Python programs.
- Evaluation is based on few-shot prompting with specific model families.

Conclusion

Takeaway

PAL improves reasoning by making the LLM generate executable intermediate steps and delegating exact computation to a symbolic runtime.

- It keeps the strength of LLMs: natural-language understanding and decomposition.
- It avoids a key weakness of LLMs: unreliable arithmetic and step execution.
- It provides a simple but powerful neuro-symbolic pattern: **language model as planner, interpreter as executor**.

Thank you!

Questions and feedback?